

Tauri App From Scratch: Setup Guide (Tauri v2)

Covers: desktop setup, frontend frameworks (Next.js, SvelteKit, Nuxt, Vite), Windows installers, Android APK export, iOS builds (Mac required), GitHub Actions auto-builds. **Checked against:** the official Tauri v2 docs (Prerequisites, Create a Project, CLI reference, Windows Installer, Android and iOS Code Signing, App Store distribution, GitHub pipeline, and the Next.js, SvelteKit, Nuxt and Vite framework pages) and the `tauri-action` workflow example, on 3 Oct 2026. The Qwik, Leptos and Trunk setups were not checked; follow their official pages. I can't run iOS builds myself (they need a Mac), so treat section 10 as documented but untested. Anything else I could not verify is listed in section 15, and section 14 is a step-by-step process for debugging both known and unknown errors. If something breaks later, the official docs win: <https://v2.tauri.app>

1. Install prerequisites

Windows

1. **Microsoft C++ Build Tools:** <https://visualstudio.microsoft.com/visual-cpp-build-tools/> and tick "**Desktop development with C++**".
2. **WebView2:** already on Windows 10 (1803+) and Windows 11. On older systems install the "Evergreen Bootstrapper" from Microsoft's WebView2 page.
3. **Rust:** install `rustup-init.exe` from <https://rustup.rs>. Make sure the **MSVC** toolchain is the default. If Rust is already installed, run:

```
rustup default stable-msvc
```

4. **Node.js LTS:** <https://nodejs.org>
5. **VBSCRIPT (only for building MSI installers):** normally enabled already. If an MSI build fails with `failed to run light.exe`, go to **Settings > Apps > Optional features > More Windows features** and make sure **VBSCRIPT** is ticked.
6. **Restart your terminal** (sometimes the PC) so PATH updates apply.

macOS

```
xcode-select --install          # enough for desktop apps
curl --proto '=https' --tlsv1.2 https://sh.rustup.rs -sSf | sh
```

Then install Node.js LTS. (Full Xcode is only needed for iOS; see section 10.)

Linux (Debian/Ubuntu)

```
sudo apt update
sudo apt install libwebkit2gtk-4.1-dev build-essential curl wget file \
  libxdo-dev libssl-dev libayatana-appindicator3-dev librsvg2-dev
curl --proto '=https' --tlsv1.2 https://sh.rustup.rs -sSf | sh
```

Other distros: <https://v2.tauri.app/start/prerequisites/>

Verify

```
rustc --version
cargo --version
node --version
npm --version
```

All four must print a version. If not, reopen the terminal.

Optional: want pnpm or yarn? Run `corepack enable` once.

2. Create the project

Easiest way:

```
npm create tauri-app@latest
```

(Official alternatives: `sh <(curl https://create.tauri.app/sh)` on macOS/Linux, or `irm https://create.tauri.app/ps | iex` in PowerShell.)

Prompts: project name, identifier (e.g. `com.yourname.myapp`), frontend language, package manager, UI template (vanilla, React, Vue, Svelte, Solid, Angular, Preact, Yew, Leptos, Sycamore).

```
cd my-app
npm install
```

Using Next.js, SvelteKit, Nuxt or another framework? See section 13.

Already have a frontend? Add Tauri to it instead:

```
npm install -D @tauri-apps/cli@latest
npm run tauri init
```

Answer the prompts (app name, web assets folder, dev server URL like `http://localhost:5173`, dev command, build command). For Vite, also add this to `vite.config.ts` so it ignores Rust files:

```
server: { watch: { ignored: ["/src-tauri/"] } }
```

3. Run it (dev mode)

```
npm run tauri dev
```

- The first run is slow (Rust compiles every dependency once). Later runs are fast.
- Frontend edits hot-reload; Rust edits trigger a rebuild.

Health check anytime:

```
npm run tauri info
```

It lists your OS, Rust, Node, WebView2/Android setup and flags problems. Run this first when anything misbehaves. For the full step-by-step process, see section 14.

4. Project structure

```

my-app/
├─ src/                # Frontend
├─ src-tauri/
│  └─ src/
│     └─ main.rs       # Entry point
│     └─ lib.rs        # Your Rust commands + app setup
│  └─ capabilities/    # Permissions your app may use
│  └─ icons/           # App icons
│  └─ Cargo.toml       # Rust dependencies
│  └─ tauri.conf.json  # App name, window, build, bundle settings
├─ package.json
└─ vite.config.*       # Frontend bundler config (if using Vite)

```

5. Call Rust from your frontend

`src-tauri/src/lib.rs`

```

#[tauri::command]
fn greet(name: &str) -> String {
    format!("Hello, {}!", name)
}

#[cfg_attr(mobile, tauri::mobile_entry_point)]
pub fn run() {
    tauri::Builder::default()
        .invoke_handler(tauri::generate_handler![greet])
        .run(tauri::generate_context!())
        .expect("error while running tauri application");
}

```

Frontend

```

import { invoke } from "@tauri-apps/api/core";
const msg = await invoke<string>("greet", { name: "World" });

```

Every new command must be listed in `generate_handler![...]`.

6. Plugins and permissions

```

npm run tauri add fs      # file system
npm run tauri add dialog  # open/save dialogs
npm run tauri add store   # key-value storage
npm run tauri add sql     # SQL databases

```

Full list: <https://v2.tauri.app/plugin/>

Tauri v2 blocks features until you allow them. When you add a plugin with `npm run tauri add <plugin>`, the CLI **automatically adds that plugin's default permission** to your capabilities (per the official permissions docs). You only need to add permissions yourself for anything beyond the default, in `src-tauri/capabilities/default.json` (add to the existing `permissions` array, keep the rest of the file):

```

"permissions": ["core:default", "fs:default", "dialog:default"]

```

Or let the CLI do it:

```
npm run tauri permission add fs:default
npm run tauri permission ls          # list available permissions
```

A "not allowed" error from a plugin almost always means a missing permission. If you installed a plugin by hand instead of with `tauri add`, add its permission yourself.

7. Configure the app

Edit `src-tauri/tauri.conf.json`: `productName`, `version`, `identifier`, window size/title (`app.windows`), and `bundle` settings.

App icon (square PNG, or SVG with transparency):

```
npm run tauri icon path/to/app-icon.png
```

It generates icons for every platform, including Android.

8. Build a desktop installer

```
npm run tauri build
```

Output goes to `src-tauri/target/release/bundle/`:

- Windows: `.msi` and `-setup.exe` (NSIS)
- macOS: `.app` and `.dmg`
- Linux: `.deb`, `.rpm`, `.AppImage`

You normally build for the OS you're on. For all three, use GitHub Actions (section 12).

9. Export an Android APK

Do this after section 3 works on desktop.

9.1 Install Android prerequisites

1. Install **Android Studio**: <https://developer.android.com/studio> (open it once to finish setup).
2. In Android Studio: **SDK Manager** (Settings > Languages & Frameworks > Android SDK) and install:
 - Android SDK Platform
 - Android SDK Platform-Tools
 - **NDK (Side by side)**
 - Android SDK Build-Tools
 - Android SDK Command-line Tools (latest)
3. Set environment variables, then restart your terminal.

Windows (PowerShell):

```
[System.Environment]::SetEnvironmentVariable("JAVA_HOME", "C:\Program Files\Android\Android Studio\jbr", "User")
[System.Environment]::SetEnvironmentVariable("ANDROID_HOME", "$env:LocalAppData\Android\Sdk", "User")
$VERSION = Get-ChildItem -Name "$env:LocalAppData\Android\Sdk\ndk" | Select-Object -Last 1
[System.Environment]::SetEnvironmentVariable("NDK_HOME", "$env:LocalAppData\Android\Sdk\ndk\$VERSION", "User")
```

To refresh the current PowerShell session without restarting:

```
[System.Environment]::GetEnvironmentVariables("User").GetEnumerator() | % { Set-Item -Path "Env:\${_.key}" -Value $_.value }
```

Linux:

```
export JAVA_HOME=/opt/android-studio/jbr      # path depends on your distro
export ANDROID_HOME="$HOME/Android/Sdk"
export NDK_HOME="$ANDROID_HOME/ndk/${ls -1 $ANDROID_HOME/ndk}"
```

macOS:

```
export JAVA_HOME="/Applications/Android Studio.app/Contents/jbr/Contents/Home"
export ANDROID_HOME="$HOME/Library/Android/sdk"
export NDK_HOME="$ANDROID_HOME/ndk/${ls -1 $ANDROID_HOME/ndk}"
```

4. Add the Rust Android targets:

```
rustup target add aarch64-linux-android armv7-linux-androideabi i686-linux-android x86_64-linux-android
```

5. **Windows:** turn on **Developer Mode** (Settings > System > For developers) so symlinks work.

9.2 Initialize Android in your project

```
npm run tauri android init
```

This creates `src-tauri/gen/android/`. Commit that folder to Git (but never the keystore files, see 9.5).

9.3 Test on a phone or emulator

- **Phone:** enable Developer Options + USB debugging, plug in, accept the prompt.
- **Emulator:** create one in Android Studio's Device Manager.

```
npm run tauri android dev
```

Handy flags: `--open` (open Android Studio instead), `--host` (use your PC's network address, needed when testing on a physical phone; your phone and PC must be on the same Wi-Fi). When `--host` is used the CLI sets `TAURI_DEV_HOST`, which your frontend dev server must listen on. The official templates already handle this. On Windows the public network address is used by default, so allow the dev server through the firewall if the phone can't connect.

9.4 Build the APK

```
# Debug APK: installable right away (signed with the debug key)
npm run tauri android build -- --debug --apk

# Release APK: smaller, but must be signed before it will install (see 9.5)
npm run tauri android build -- --apk

# One CPU type only = smaller/faster (most modern phones are aarch64)
npm run tauri android build -- --apk --target aarch64

# Separate APK per CPU type
npm run tauri android build -- --apk --split-per-abi
```

Valid `--target` values: `aarch64`, `armv7`, `i686`, `x86_64`. By default all four are built.

APKs end up under `src-tauri/gen/android/app/build/outputs/apk/` (the CLI prints the path when it finishes). An unsigned release build has "unsigned" in the file name.

9.5 Sign a release APK

Android won't install unsigned release APKs. Make a keystore once and **back it up** (lose it and you can't update your app under the same signature).

1. **Create the keystore** (stores it in your home folder):

```
keytool -genkey -v -keystore ~/upload-keystore.jks -keyalg RSA -keysize 2048 -validity 10000 -alias upload
```

If `keytool` isn't found, use the one inside Android Studio's JDK, e.g. `/opt/android-studio/jbr/bin/keytool` (Linux) or the `jbr\bin\keytool.exe` under your Android Studio folder (Windows).

2. **Create `src-tauri/gen/android/keystore.properties`** :

```
password=<the password you chose in keytool>
keyAlias=upload
storeFile=<full path to the keystore, e.g. /Users/you/upload-keystore.jks or C:\\Users\\you\\upload-keystore.jks>
```

Keep this file and the `.jks` private. Never commit them (add both to `.gitignore`).

3. **Edit `src-tauri/gen/android/app/build.gradle.kts`** (the one inside the `app/` folder, the one that has a `buildTypes` block).

- Add at the very top: `import java.io.FileInputStream`
- Add this **before** the `buildTypes` block (inside `android { ... }`):

```
signingConfigs {
    create("release") {
        val keystorePropertiesFile = rootProject.file("keystore.properties")
        val keystoreProperties = Properties()
        if (keystorePropertiesFile.exists()) {
            keystoreProperties.load(FileInputStream(keystorePropertiesFile))
        }

        keyAlias = keystoreProperties["keyAlias"] as String
        keyPassword = keystoreProperties["password"] as String
        storeFile = file(keystoreProperties["storeFile"] as String)
        storePassword = keystoreProperties["password"] as String
    }
}
```

- Inside `buildTypes`, in the `release` block, add:

```
getByName("release") {
    signingConfig = signingConfigs.getByName("release")
}
```

If `Properties` shows as unresolved, add `import java.util.Properties` at the top as well.

4. **Rebuild:**

```
npm run tauri android build -- --apk
```

The release APK is now signed automatically.

Official guide: <https://v2.tauri.app/distribute/sign/android/>

9.6 Install the APK

- Copy the `.apk` to your phone and tap it (allow "Install unknown apps" for your file manager when asked).
- Or, with the phone plugged in: `adb install path/to/app.apk`

9.7 Android notes and fixes

- **Google Play** needs an AAB, not an APK: `npm run tauri android build -- --aab`.
- **Permissions** like camera or storage need entries in `AndroidManifest.xml` (under `src-tauri/gen/android/app/src/main/`) and Tauri capabilities.
- Some plugins are desktop-only (global shortcuts, tray, window controls).
- The first Android build is very slow (Gradle downloads a lot).

Problem	Fix
<code>JAVA_HOME</code> / <code>ANDROID_HOME</code> / <code>NDK_HOME</code> errors	Re-check 9.1 step 3, restart terminal, then run <code>npm run tauri info</code>
Android target / linker errors	Run the <code>rustup target add ...</code> command from 9.1 step 4
Symlink or permission errors (Windows)	Enable Developer Mode (9.1 step 5)
Phone not detected	<code>adb devices</code> ; re-enable USB debugging and accept the prompt
Blank screen on phone in dev	Dev server unreachable: same Wi-Fi, use <code>--host</code> , check firewall; or test a built APK
"App not installed"	Release APK isn't signed, or an older build with a different signature is installed (uninstall it first)
Weird Gradle errors	<code>cd src-tauri/gen/android</code> then <code>./gradlew clean</code> (<code>gradlew.bat clean</code> on Windows)

10. iOS: build and run (needs a Mac)

10.1 Read this first

- **iOS builds only work on macOS with full Xcode.** You cannot build for iOS on Windows or Linux. Your options are a Mac, a rented cloud Mac, or a macOS machine in CI (GitHub Actions, see 10.9).
- **What you need depends on your goal:**

Goal	What you need
Run in the iOS Simulator	A Mac, Xcode, and an Apple ID signed in to Xcode (free)
Run on your own iPhone	The above, plus a cable. A free Apple ID generally works for local testing, but Apple limits it (builds expire quickly, limited devices). This is general Apple behavior, not from the Tauri docs
TestFlight, App Store, or ad-hoc distribution	Apple Developer Program membership (the Tauri docs say it costs \$99/year at time of writing)

10.2 Install prerequisites (on the Mac)

1. **Install full Xcode** from the Mac App Store or the Apple Developer website, then **launch it once** so it finishes setup. Xcode Command Line Tools alone are **not** enough for iOS.
2. In **Xcode > Settings**: sign in with your Apple ID (Accounts tab), and make sure an iOS platform/Simulator runtime is installed (the menu labels vary between Xcode versions).
3. Make sure the command line tools point at Xcode (general macOS check):

```
xcodeselect -p
# should print /Applications/Xcode.app/Contents/Developer
# if not:
sudo xcodeselect -s /Applications/Xcode.app/Contents/Developer
```

4. **Add the Rust iOS targets:**

```
rustup target add aarch64-apple-ios x86_64-apple-ios aarch64-apple-ios-sim
```

`aarch64-apple-ios` is for real iPhones/iPads, `aarch64-apple-ios-sim` for the Simulator on Apple Silicon Macs, and `x86_64-apple-ios` for the Simulator on Intel Macs.

5. **Install CocoaPods** (needs Homebrew, <https://brew.sh>):

```
brew install cocoapods
```

6. **Check it:**

```
npm run tauri info
```

Look at the iOS part of the output (it lists your Apple developer teams once Xcode is signed in).

Official page: <https://v2.tauri.app/start/prerequisites/>

10.3 Initialize iOS in your project

```
npm run tauri ios init
```

This creates `src-tauri/gen/apple/` (the Xcode project). Commit that folder to Git, but never commit certificates, `.p8` keys or other secrets.

If you get "No code signing certificates found": the CLI needs your Apple **Team ID**. Any one of these works (these come from Tauri community discussions, not the official docs):

- Set it in `src-tauri/tauri.conf.json`:

```
{ "bundle": { "iOS": { "developmentTeam": "YOURTEAMID" } } }
```

- Or set the environment variable for the command: `APPLE_DEVELOPMENT_TEAM=YOURTEAMID npm run tauri ios init` (older posts call it `TAURI_APPLE_DEVELOPMENT_TEAM`; it was renamed).
- Or open `src-tauri/gen/apple/*.xcodeproj` in Xcode and pick your Team under **Signing & Capabilities**.

Your Team ID shows in `npm run tauri info` and on the Membership page of developer.apple.com.

10.4 Run on the Simulator or an iPhone

```
npm run tauri ios dev
```

Pick a simulator or device when prompted (or pass its name). Useful flags: `--open` (open the project in Xcode), `--host` (use your Mac's network address so a physical iPhone can reach the dev server; the phone and Mac must be on the same Wi-Fi).

Physical iPhone checklist (general Apple requirements):

1. Plug it in and tap **Trust** on the phone.
2. Turn on **Developer Mode** on the phone (Settings > Privacy & Security > Developer Mode) and restart it.
3. In Xcode, select your Team under Signing & Capabilities so it can create a provisioning profile.
4. With a free Apple ID you may also need to trust the developer profile on the phone (Settings > General > VPN & Device Management).

The frontend dev server must listen on `TAURI_DEV_HOST`; the Vite config in section 13.7 already does this.

10.5 Build an .ipa

```
npm run tauri ios build -- --debug                # debug build
npm run tauri ios build -- --export-method debugging # for registered test devices
npm run tauri ios build -- --export-method app-store-connect # for TestFlight / App Store
npm run tauri ios build -- --open                  # open in Xcode to archive/distribute there
npm run tauri ios build -- --build-number 7        # append a build number to the version
```

The `.ipa` ends up at:

```
src-tauri/gen/apple/build/arm64/<AppName>.ipa
```

The accepted `--export-method` names depend on your Tauri CLI and Xcode versions. Check yours with `npm run tauri ios build -- --help`. The Tauri signing docs also describe an "ad-hoc" method (for registered devices) alongside `debugging` and `app-store-connect`.

An **.ipa is not like an APK**. You can't just tap it to install. Use `ios dev` / Xcode to run on your own device, and TestFlight for testers.

10.6 Signing: what to pick

Apple requires code signing. The easy route is **automatic signing**: Xcode registers your bundle ID and manages certificates and profiles for you. **Manual signing** means creating a certificate and provisioning profile yourself.

Export method	Certificate type
<code>debugging</code>	Apple Development / iOS App Development
<code>app-store-connect</code>	Apple Distribution / iOS Distribution
<code>ad-hoc</code>	Apple Distribution / iOS Distribution

Your bundle ID registered in App Store Connect must match `identifier` in `tauri.conf.json`. Official guide: <https://v2.tauri.app/distribute/sign/ios/>

10.7 Upload to App Store Connect / TestFlight

1. Register your app in **App Store Connect** (Bundle ID = your `identifier`).
2. Optional: set a build version in `tauri.conf.json`:

```
{ "bundle": { "iOS": { "bundleVersion": "100" } } }
```

3. Read the official note about **encryption export compliance** on the App Store page. It involves an entry in your iOS `Info.plist`; the CLI merges `src-tauri/Info.ios.plist` into the app's `Info.plist`.
4. Build for the store:

```
npm run tauri ios build -- --export-method app-store-connect
```

5. Upload with an **App Store Connect API key** (App Store Connect > Users and Access > Integrations; the docs say to give it Admin access):

```
xcrun altool --upload-app --type ios \  
  --file "src-tauri/gen/apple/build/arm64/$APPNAME.ipa" \  
  --apiKey $APPLE_API_KEY_ID --apiIssuer $APPLE_API_ISSUER
```

Replace `$APPNAME` with your app's name. (Apple's Transporter app is a manual alternative.)

6. In App Store Connect, add testers via TestFlight or submit for review.

iOS icons: run the icon command after `ios init`. iOS icons can't be transparent, so give the generator a background color (quote the `#` so your shell doesn't treat it as a comment):

```
npm run tauri icon app-icon.png -- --ios-color "#ffffff"
```

Official page: <https://v2.tauri.app/distribute/app-store/>

10.8 Debug on iOS

- **Rust logs and errors:** the terminal running `ios dev`, and Xcode's console if you used `--open`.
- **WebView console:** on the Mac, enable Safari's developer features (Safari > Settings > Advanced > "Show features for web developers"), then use the **Develop** menu to pick your Simulator or iPhone and inspect the app's webview. This is standard Apple tooling, not Tauri-specific.
- Everything in the process in section 14.2 applies too.

10.9 iOS in GitHub Actions (macOS runner)

This is **my own composition**, not an official Tauri example, and I haven't run it. Get a successful signed build on a Mac first, then automate. For CI, the Tauri docs say automatic signing needs an App Store Connect API key exposed as `APPLE_API_ISSUER`, `APPLE_API_KEY` and `APPLE_API_KEY_PATH`.

Add repo secrets: `APPLE_DEVELOPMENT_TEAM`, `APPLE_API_ISSUER`, `APPLE_API_KEY` (the key ID), and `APPLE_API_KEY_P8` (the full contents of the `.p8` file). Commit `src-tauri/gen/apple/`.

`.github/workflows/ios.yml`:

```
name: iOS build

on:
  workflow_dispatch:
  push:
    tags:
      - 'app-v*'

jobs:
  ios:
    runs-on: macos-latest
    steps:
      - uses: actions/checkout@v7

      - uses: actions/setup-node@v6
        with:
          node-version: lts/*
          cache: 'npm'

      - uses: dtolnay/rust-toolchain@stable
        with:
          targets: aarch64-apple-ios,x86_64-apple-ios,aarch64-apple-ios-sim

      - uses: swatinem/rust-cache@v2
        with:
          workspaces: './src-tauri -> target'

      - run: npm install

      - name: Write App Store Connect API key
        run: |
          mkdir -p "$RUNNER_TEMP/keys"
          echo "${{ secrets.APPLE_API_KEY_P8 }}" > "$RUNNER_TEMP/keys/AuthKey_${{ secrets.APPLE_API_KEY }}.p8"

      - name: Build IPA
        env:
          APPLE_DEVELOPMENT_TEAM: "${{ secrets.APPLE_DEVELOPMENT_TEAM }}"
          APPLE_API_ISSUER: "${{ secrets.APPLE_API_ISSUER }}"
          APPLE_API_KEY: "${{ secrets.APPLE_API_KEY }}"
          APPLE_API_KEY_PATH: "${{ runner.temp }}/keys/AuthKey_${{ secrets.APPLE_API_KEY }}.p8"
        run: npm run tauri ios build -- --export-method app-store-connect

      - uses: actions/upload-artifact@v4
        with:
          name: app-ipa
          path: src-tauri/gen/apple/build/arm64/*.ipa
```

Things to watch: the runner's Xcode version decides which `--export-method` names work; CocoaPods is usually preinstalled on GitHub's macOS images, but if you see `pod: command not found`, add a `brew install cocoapods` step; and CI signing is the most failure-prone part, so expect to iterate.

10.10 iOS troubleshooting

Problem	Fix
Trying to build on Windows/Linux	Not possible. Use a Mac, a cloud Mac, or a macOS CI runner
"No code signing certificates found" at <code>ios init</code>	Set the Team ID (10.3) and make sure Xcode is signed in with your Apple ID
<code>pod: command not found</code> / CocoaPods errors	<code>brew install cocoapods</code> , then reopen the terminal
"No iOS SDK installed" / no simulator	Install an iOS platform runtime in Xcode Settings
Xcode tools point to Command Line Tools	<code>xcode-select -p</code> , then <code>sudo xcode-select -s /Applications/Xcode.app/Contents/Developer</code>
<code>exportOptionsPlist</code> error for key "method"	The export-method name isn't accepted by your Xcode/CLI combo. Update Xcode and the Tauri CLI, and check <code>ios build --help</code> . Reported in tauri-apps/tauri issue #13818; I haven't confirmed the cause
"Missing code-signing certificate" or a distribution profile error on <code>app-store-connect</code>	Distribution signing isn't set up: use automatic signing (with the API key in CI) or provide a distribution certificate and profile. A similar report (issue #11092) required editing <code>ExportOptions.plist</code> at the time; it may be fixed in newer versions
App is blank on a real iPhone in dev	Use <code>--host</code> , same Wi-Fi, and make sure your dev server listens on <code>TAURI_DEV_HOST</code>
Phone says "Untrusted Developer"	Trust the profile in Settings > General > VPN & Device Management (free Apple ID builds)

11. Windows: build and ship an installer

11.1 Build

```
npm run tauri build
```

Outputs in `src-tauri/target/release/bundle/`:

- `nsis/` : `YourApp_x.y.z_x64-setup.exe` (the usual one to share)
- `msi/` : `YourApp_x.y.z_x64_en-US.msi` (good for IT/company deployment)

MSI files can **only be built on Windows** (they use WiX Toolset v3). Build just one format to save time, in `tauri.conf.json`:

```
{ "bundle": { "targets": ["nsis"] } }
```

or from the command line: `npm run tauri build -- --bundles nsis` (or `msi`).

11.2 Install mode (per-user vs all-users)

By default the installer installs for the **current user only** (no admin needed, goes in `%LOCALAPPDATA%`). To install system-wide in `Program Files`:

```
{ "bundle": { "windows": { "nsis": { "installMode": "perMachine" } } } }
```

Use `"both"` to let the user choose (needs admin rights).

11.3 WebView2 on other people's PCs

Windows 10 (April 2018+) and 11 already include WebView2. For others, pick how the installer gets it, in `tauri.conf.json` under `bundle > windows > webviewInstallMode > type`:

type	Needs internet?	Extra installer size	Notes
downloadBootstrapper	Yes	0 MB	Default
embedBootstrapper	Yes	~1.8 MB	Better for Windows 7 MSI
offlineInstaller	No	~127 MB	For offline PCs
fixedVersion (config type fixedRuntime, needs a path)	No	~180 MB	You ship a fixed WebView2 version
skip	No	0 MB	Not recommended; app won't run without WebView2

Example:

```
{ "bundle": { "windows": { "webviewInstallMode": { "type": "embedBootstrapper" } } } }
```

11.4 32-bit and ARM64 builds

```
# 32-bit
rustup target add i686-pc-windows-msvc
npm run tauri build -- --target i686-pc-windows-msvc

# ARM64 (also install "MSVC ... C++ ARM64 build tools" via Visual Studio Installer > Modify > Individual components)
rustup target add aarch64-pc-windows-msvc
npm run tauri build -- --target aarch64-pc-windows-msvc
```

Note: an ARM64 NSIS installer is itself x86 (runs under emulation), but the app inside is native ARM64.

11.5 Build a Windows installer from Linux/macOS (last resort)

Possible for **NSIS only**, with caveats. It needs NSIS, LLVM/LLD, the `x86_64-pc-windows-msvc` Rust target and `cargo-xwin`, then:

```
npm run tauri build -- --runner cargo-xwin --target x86_64-pc-windows-msvc
```

GitHub Actions (section 12) is usually easier. Details: <https://v2.tauri.app/distribute/windows-installer/>

11.6 The SmartScreen warning

Unsigned installers show "Windows protected your PC / Unknown publisher". Removing it needs a code-signing certificate or signing service: <https://v2.tauri.app/distribute/sign/windows/>. For personal use or friends, click **More info > Run anyway**.

11.7 Windows gotchas

- Keep the project path short (e.g. `C:\dev\my-app`) to avoid long-path errors.
- Bump the version in `tauri.conf.json` (and `package.json` / `Cargo.toml` to match) before each release.
- If the MSI build rejects your version string, use a plain `x.y.z`.
- failed to run `light.exe` means check VBSCRIPT (section 1, Windows step 5).
- `link.exe not found` means C++ Build Tools are missing.

12. GitHub Actions: automatic builds and releases

Builds Windows, macOS and Linux installers on GitHub's servers, so you don't need a Mac or Linux PC.

12.1 One-time GitHub setup

1. Create a repo and push your project:

```
git init
git add .
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/YOUR_USER/YOUR_REPO.git
git push -u origin main
```

2. **Settings > Actions > General > Workflow permissions > "Read and write permissions" > Save.** Without this, releases fail with "Resource not accessible by integration".
3. Make sure `.gitignore` excludes `node_modules`, `src-tauri/target`, and keystore/secret files.
4. Commit your lockfile (`package-lock.json`, etc.).

12.2 Desktop release workflow

Create `.github/workflows/release.yml`. This follows the official Tauri example, triggered by a version tag instead of a branch:

```
name: 'publish'

on:
  workflow_dispatch:
  push:
    tags:
      - 'app-v*'

jobs:
  publish-tauri:
    permissions:
      contents: write
    strategy:
      fail-fast: false
    matrix:
      include:
        - platform: 'macos-latest' # Apple Silicon Macs
          args: '--target aarch64-apple-darwin'
        - platform: 'macos-latest' # Intel Macs
          args: '--target x86_64-apple-darwin'
        - platform: 'ubuntu-24.04' # ubuntu-22.04 runners are being retired (see section 15)
          args: ''
        - platform: 'windows-latest'
          args: ''

    runs-on: ${ matrix.platform }
    steps:
      - uses: actions/checkout@v7

      - name: install dependencies (ubuntu only)
        if: matrix.platform == 'ubuntu-24.04'
        run: |
          sudo apt-get update
          sudo apt-get install -y libwebkit2gtk-4.1-dev libayatana-appindicator3-dev librsvg2-dev patchelf xdg-utils

      - name: setup node
        uses: actions/setup-node@v6
        with:
          node-version: lts/*
          cache: 'npm' # change to yarn or pnpm if you use those

      - name: install Rust stable
        uses: dtolnay/rust-toolchain@stable
        with:
```

```

    targets: ${{ matrix.platform == 'macos-latest' && 'aarch64-apple-darwin,x86_64-apple-darwin' || '' }}

- name: Rust cache
  uses: swatinem/rust-cache@v2
  with:
    workspaces: './src-tauri -> target'

- name: install frontend dependencies
  run: npm install # or yarn / pnpm install

- uses: tauri-apps/tauri-action@v1
  env:
    GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
  with:
    tagName: app-v__VERSION__ # replaced automatically with your app version
    releaseName: 'App v__VERSION__'
    releaseBody: 'See the assets to download this version and install.'
    releaseDraft: true
    prerelease: false
    args: ${{ matrix.args }}

```

Notes:

- The official example triggers on pushes to a `release` branch instead. To use that, replace the `tags:` block with `branches: [ release ]`.
- **Runner label:** Tauri's official example still says `ubuntu-22.04`, but GitHub began deprecating Ubuntu 22.04 runners on 17 Sep 2026 and retires them on 17 Apr 2027, and recommends `ubuntu-24.04`, `ubuntu-26.04` or `ubuntu-latest`. This guide pins `ubuntu-24.04`. (`ubuntu-latest` is announced to switch to 26.04 in Nov 2026.) The apt package list follows Tauri's Debian prerequisites; if `apt` can't find one on your runner, compare with <https://v2.tauri.app/start/prerequisites/>. Linux apps built on a newer Ubuntu may not run on older distros.
- Linux ARM64 builds (`ubuntu-24.04-arm`, previously `ubuntu-22.04-arm`) are available on **public** repos only. Add it to the matrix (and to the `if:` line) if you want them.
- **macOS without an Apple certificate:** use an ad-hoc signing identity so Apple Silicon builds downloaded from GitHub aren't flagged as "damaged": <https://v2.tauri.app/distribute/sign/macos/#ad-hoc-signing>
- Your app isn't in the repo root? Add `projectPath: path/to/app` under `with:`.

12.3 Cut a release

The tag you push must match `app-v` + your app version:

```

# 1. bump the version in tauri.conf.json, package.json and Cargo.toml (say, to 1.0.0)
git add .
git commit -m "release 1.0.0"
git tag app-v1.0.0
git push origin main --tags

```

Watch the **Actions** tab. When finished, open **Releases**, check the **draft**, then **Publish**. The installers are attached.

12.4 Android APK workflow

Prerequisite: you ran `npm run tauri android init` locally and **committed** `src-tauri/gen/android/`.

Create `.github/workflows/android.yml`:

```

name: Android APK

on:
  workflow_dispatch:
  push:
    tags:
      - 'app-v*'

jobs:
  apk:
    runs-on: ubuntu-24.04
    steps:
      - uses: actions/checkout@v7

      - uses: actions/setup-java@v5
        with:
          distribution: temurin
          java-version: 17

      - uses: actions/setup-node@v6
        with:
          node-version: lts/*
          cache: 'npm'

      - uses: dtolnay/rust-toolchain@stable
        with:
          targets: aarch64-linux-android,armv7-linux-androideabi,i686-linux-android,x86_64-linux-android

      - uses: swatinem/rust-cache@v2
        with:
          workspaces: './src-tauri -> target'

      - name: Point NDK_HOME at the runner's NDK
        run: echo "NDK_HOME=$ANDROID_NDK_LATEST_HOME" >> $GITHUB_ENV

      - run: npm install

      - name: Build debug APK (installable, debug-signed)
        run: npm run tauri android build -- --debug --apk

      - uses: actions/upload-artifact@v4
        with:
          name: app-apk
          path: src-tauri/gen/android/app/build/outputs/apk/**/*.apk

```

After it runs, open the run in the **Actions** tab and download **app-apk** under **Artifacts**.

Action versions move on. If GitHub warns that one is outdated, bump to that action's latest major version.

For a signed release APK in CI (uses the official approach from Tauri's Android signing guide):

1. Complete 9.5 steps 1 to 3 locally and commit the edited `build.gradle.kts` (not the keystore or `keystore.properties`).
2. Base64 your keystore: `base64 -w0 upload-keystore.jks` (macOS: `base64 -i upload-keystore.jks`).
3. In **Settings > Secrets and variables > Actions**, add secrets: `ANDROID_KEY_ALIAS` (`upload`), `ANDROID_KEY_PASSWORD` , `ANDROID_KEY_BASE64` .
4. Insert this step before the build step:

```
- name: setup Android signing
  run: |
    cd src-tauri/gen/android
    echo "keyAlias=${{ secrets.ANDROID_KEY_ALIAS }}" > keystore.properties
    echo "password=${{ secrets.ANDROID_KEY_PASSWORD }}" >> keystore.properties
    base64 -d <<< "${{ secrets.ANDROID_KEY_BASE64 }}" > $RUNNER_TEMP/keystore.jks
    echo "storeFile=$RUNNER_TEMP/keystore.jks" >> keystore.properties
```

5. Change the build command to `npm run tauri android build -- --apk`.

iOS in CI: see section 10.9 (needs a macOS runner).

12.5 CI troubleshooting

Problem	Fix
"Resource not accessible by integration"	Set workflow permissions to Read and write (12.1 step 2)
Workflow doesn't start	Tag must match <code>app-v*</code> (e.g. <code>app-v1.0.0</code>) and be pushed (<code>git push --tags</code>)
Cache step or install fails	Commit your lockfile, and make sure <code>cache:</code> and the install command match your package manager
Linux job can't find webkit or another package	Keep the <code>apt-get install</code> step; use <code>libwebkit2gtk-4.1-dev</code> (4.0 is for Tauri v1) and compare with the prerequisites page
"No runner matching ubuntu-22.04" or deprecation warnings	Change the label to <code>ubuntu-24.04</code> (section 15)
Android job: NDK not found	Check the "Point NDK_HOME" step and the build log; runner NDK paths can change
Android job: <code>gen/android</code> missing	Commit it, or add a step running <code>npm run tauri android init --ci</code> before the build
macOS app says "damaged"	Use ad-hoc signing (see 12.2 notes) or Apple signing/notarization
Builds are slow	Normal the first time; the Rust cache speeds up later runs. Watch GitHub's usage limits on private repos

Reference examples: <https://github.com/tauri-apps/tauri-action> and <https://v2.tauri.app/distribute/pipelines/github/>

13. Frontend frameworks (React, Vue, Svelte, Next.js, SvelteKit, Nuxt)

13.1 The one rule

Tauri loads **static files** (HTML, CSS, JS) into a webview. There is **no Node.js server** at runtime. So your framework must build to static output (SSG, or a single-page app). Anything that needs a server (server-side rendering, API routes, server endpoints, server actions) will not work. Move that logic into Rust commands (section 5).

13.2 Easiest path: pick a template

`npm create tauri-app@latest` offers pre-configured templates for vanilla, Vue, Svelte, React, SolidJS, Angular, Preact, Yew, Leptos and Sycamore. If your framework is on that list, pick it and you can skip the rest of this section.

13.3 Adding Tauri to Next.js, SvelteKit, Nuxt, or any existing frontend

1. Create the framework project the way its own docs say (e.g. `create-next-app`, the Svelte CLI, the Nuxt installer) and make sure it runs on its own.
2. From the project root, add the Tauri CLI and API, and add a `tauri` script to `package.json`:

```
npm install -D @tauri-apps/cli@latest
npm install @tauri-apps/api
```

```
"scripts": { "tauri": "tauri" }
```

(Keep your existing `dev` and `build` scripts.)

3. Initialize the Rust side:

```
npx tauri init
```

Answer the prompts using the table below (the web assets path is relative to `src-tauri/`).

4. Apply the framework config from 13.4 to 13.7.

5. Run it:

```
npm run tauri dev
```

Quick reference (all use `"beforeDevCommand": "npm run dev"`):

Framework	devUrl	frontendDist	beforeBuildCommand	Must-have setting
Next.js	<code>http://localhost:3000</code>	<code>../out</code>	<code>npm run build</code>	<code>output: 'export'</code>
Nuxt	<code>http://localhost:3000</code>	<code>../dist</code>	<code>npm run generate</code>	<code>ssr: false</code>
SvelteKit	<code>http://localhost:5173</code>	<code>../build</code>	<code>npm run build</code>	<code>adapter-static</code> with <code>fallback: 'index.html'</code>
Vite (React, Vue, Svelte, Solid...)	<code>http://localhost:5173</code> (must match your Vite port)	<code>../dist</code>	<code>npm run build</code>	<code>strictPort: true</code> , host settings below

These all go in `src-tauri/tauri.conf.json` under `"build"`:

```
{
  "build": {
    "beforeDevCommand": "npm run dev",
    "beforeBuildCommand": "npm run build",
    "devUrl": "http://localhost:3000",
    "frontendDist": "../out"
  }
}
```

Using pnpm or yarn? Replace `npm run` with `pnpm / yarn` (e.g. `pnpm dev`, `pnpm build`).

13.4 Next.js

Tauri `build` block: the Next.js row of the table above.

next.config.mjs

```
const isProd = process.env.NODE_ENV === 'production';
const internalHost = process.env.TAURI_DEV_HOST || 'localhost';

/** @type {import('next').NextConfig} */
const nextConfig = {
  // Ensure Next.js uses SSG instead of SSR
  output: 'export',
  // Needed to use the Next.js Image component in static export mode
  images: {
    unoptimized: true,
  },
  // Without this the dev server won't resolve assets properly
  assetPrefix: isProd ? undefined : `http://${internalHost}:3000`,
};

export default nextConfig;
```

The official guide was last written against Next.js 14.2.3. Newer Next.js versions may change details, so if something differs,

check Next.js's own "Static Exports" docs.

13.5 SvelteKit

```
npm install --save-dev @sveltejs/adapter-static
```

svelte.config.js

```
import adapter from '@sveltejs/adapter-static';
import { vitePreprocess } from '@sveltejs/vite-plugin-svelte';

/** @type {import('@sveltejs/kit').Config} */
const config = {
  preprocess: vitePreprocess(),
  kit: {
    adapter: adapter({
      fallback: 'index.html',
    }),
  },
};

export default config;
```

- The official guide recommends **SPA mode** (the `fallback` above, no prerendering). With prerendering, `load` functions run at build time and can't use Tauri APIs.
- If you get "window is not defined" during build, a common fix is `export const ssr = false;` in `src/routes/+layout.ts`.
- SvelteKit runs on Vite. For testing on a physical phone, add the `server` host settings from the Vite config in 13.7 to your `vite.config.ts`.

13.6 Nuxt

nuxt.config.ts

```
export default defineNuxtConfig({
  compatibilityDate: '2025-05-15',
  devtools: { enabled: true },
  // Enable SSG
  ssr: false,
  // Lets other devices (e.g. a physical phone) reach the dev server
  devServer: {
    host: '0',
  },
  vite: {
    // Better support for Tauri CLI output
    clearScreen: false,
    envPrefix: ['VITE_', 'TAURI_'],
    server: {
      // Tauri requires a consistent port
      strictPort: true,
    },
  },
  // Avoids "EMFILE: too many open files, watch" errors
  ignore: ['**/src-tauri/**'],
});
```

Use `npm run generate` as the build command (as in the table). Make sure your `package.json` has the `generate` script. Nuxt projects normally include it.

13.7 Vite (React, Vue, Svelte, Solid, Preact...)

vite.config.ts (add your framework's plugin to `plugins: []` as usual)

```
import { defineConfig } from 'vite';

const host = process.env.TAURI_DEV_HOST;

export default defineConfig({
  // don't hide Rust errors
  clearScreen: false,
  server: {
    // must match the port in devUrl (tauri.conf.json)
    port: 5173,
    // fail instead of silently switching ports
    strictPort: true,
    // set automatically by the Tauri CLI for phone testing
    host: host || false,
    hmr: host
      ? { protocol: 'ws', host, port: 1421 }
      : undefined,
    watch: {
      // don't watch the Rust folder
      ignored: ['**/src-tauri/**'],
    },
  },
  // Expose Tauri's env variables to your frontend code
  envPrefix: ['VITE_', 'TAURI_ENV_*'],
  build: {
    // Tauri uses Chromium (WebView2) on Windows, WebKit elsewhere
    target: process.env.TAURI_ENV_PLATFORM == 'windows' ? 'chrome105' : 'safari13',
    // don't minify debug builds
    minify: !process.env.TAURI_ENV_DEBUG ? 'esbuild' : false,
    sourcemap: !!process.env.TAURI_ENV_DEBUG,
  },
});
```

The official `create-tauri-app` templates use port 1420 instead of 5173. Whatever you pick, `port` here and `devUrl` in `tauri.conf.json` must match.

13.8 Qwik, Leptos, Trunk (Rust frontends) and others

These have their own official setup pages, so follow them instead of guessing:

- Qwik: <https://v2.tauri.app/start/frontend/qwik/>
- Leptos: <https://v2.tauri.app/start/frontend/leptos/>
- Trunk (Yew, Sycamore, etc.): <https://v2.tauri.app/start/frontend/trunk/>
- Overview of all frameworks: <https://v2.tauri.app/start/frontend/>

13.9 Using Tauri from your framework code

```
import { invoke } from "@tauri-apps/api/core";

const msg = await invoke<string>("greet", { name: "Next.js" });
```

Call Tauri APIs only in **browser-side** code: inside click handlers, effects, or lifecycle hooks (`useEffect`, `onMount`, etc.), and in Next.js App Router files mark components that use them with `"use client"`. During build/prerender there is no `window`, so top-level calls will fail.

13.10 Framework troubleshooting

Problem	Fix
Blank window in dev	<code>devUrl</code> doesn't match the dev server port, or the dev server isn't running. Check the terminal and <code>strictPort</code>
Blank window in the built app	<code>frontendDist</code> points to the wrong folder, or the build produced nothing there. Run your build command alone and check the output folder (<code>out</code> , <code>dist</code> or <code>build</code>)
<code>window is not defined</code>	Tauri API call runs during build/prerender. Move it into client-only code (13.9)
Build fails on API routes / server actions / server endpoints	Not supported with static output. Rewrite as Rust commands
Images broken in Next.js	Set <code>images: { unoptimized: true }</code> (13.4)
<code>EMFILE: too many open files</code>	Tell your bundler to ignore <code>src-tauri</code> (<code>ignored: ['**/src-tauri/**']</code>)
Phone can't load the dev server	Use <code>TAURI_DEV_HOST</code> as the dev server host (already in the configs above) and <code>npm run tauri android dev -- --host</code>
<code>beforeDevCommand</code> / <code>beforeBuildCommand</code> errors	Run the same command by hand first (<code>npm run dev</code> , <code>npm run build</code>) and fix errors there

In GitHub Actions (section 12) nothing special is needed: `beforeBuildCommand` builds your frontend as part of `tauri build`. For anything not in this table, use the process in section 14.2. Items I couldn't fully verify for Next.js, SvelteKit and Nuxt are listed in section 15.

14. Debugging: a process for known and unknown bugs

Most Tauri bugs are cheap to fix once you know **which layer** is failing. Work through this in order. Don't skip to guessing.

14.1 Where errors actually show up

Where	How to see it	What shows up there
Terminal running <code>tauri dev</code>	Just look at it	Rust compile errors, Rust <code>println!</code> output, panics, and your bundler's (Vite/Next/Nuxt) errors
Rust backtrace	<code>RUST_BACKTRACE=1 npm run tauri dev</code> (PowerShell: <code>\$env:RUST_BACKTRACE=1</code> first, then run the command)	Detailed stack trace when Rust panics
WebView console	Right-click inside the app window > Inspect Element	Frontend JS errors, <code>console.log</code> , failed <code>invoke</code> calls, network errors. It's Edge DevTools on Windows, Safari's inspector on macOS, WebKitGTK's inspector on Linux
Debug build of the packaged app	<code>npm run tauri build -- --debug</code>	A bundled app (in <code>src-tauri/target/debug/bundle</code>) with the dev console enabled. Run its executable from a terminal to see output (double-clicking closes the console on errors)
Android	Android Studio's Logcat, or <code>adb logcat</code>	Native and WebView logs from a phone/emulator (standard Android tooling, not Tauri-specific)
iOS	Terminal running <code>ios dev</code> , Xcode's console, Safari's Develop menu	Rust logs, native errors, and the webview console (section 10.8)
GitHub Actions	Repo > Actions > the run > click the red step	Exact failing command and its output

Print from Rust: `println!("Message from Rust: {}", msg);`

Open devtools from code (debug builds only), inside `.setup(|app| { ... })`:

```
#[cfg(debug_assertions)]
{
    let window = app.get_webview_window("main").unwrap();
    window.open_devtools();
}
```

Devtools in **release** builds need the `devtools` Cargo feature on the `tauri` dependency. Warning from the official docs: on macOS this uses a private API and blocks App Store acceptance.

14.2 The process (use it for every bug)

1. **Read the first error, not the last.** Scroll up to the *first* red line. Later errors are usually fallout from it. Copy it exactly.
2. **Name the layer.** Use the table in 14.3.
3. **Run the health check:** `npm run tauri info`. It reports your OS, Rust, Node, WebView2/Android setup and Tauri package versions, and flags missing pieces. Add `--interactive` to let it offer fixes.
4. **Check versions match.** The `@tauri-apps/api`, `@tauri-apps/cli`, the `tauri` Rust crate and plugin versions should be on the same major (ideally same minor) version. The CLI refuses to build on a mismatch (the `--ignore-version-mismatches` flag exists, but only use it if you're sure the detection is wrong). Fix with `npm update @tauri-apps/cli @tauri-apps/api` and `cargo update` inside `src-tauri`.
5. **Shrink the problem.** Run the pieces separately:
 - Does the **frontend alone** work? `npm run dev`, then open the dev URL in a normal browser.
 - Does the **Rust side alone** compile? `cd src-tauri && cargo check`.
 - Does a **fresh project** work? `npm create tauri-app@latest` in another folder. If yes, the bug is in your changes; if no, it's your environment.
6. **Bisect your changes.** Use Git: `git stash` (or `git diff`) and re-test. Undo the last change first, then the one before, until it works.
7. **Clean in order, least destructive first**, retesting after each step:
 1. Restart `npm run tauri dev`
 2. Delete your framework's cache (`.next`, `.nuxt`, `node_modules/.vite`, `.svelte-kit`)
 3. `cd src-tauri && cargo clean` (next build is slow again)
 4. Delete `node_modules` and reinstall
 5. Android only: `cd src-tauri/gen/android && ./gradlew clean`
8. **Search the exact message.** Put the exact error text in quotes plus `tauri`. Look at Tauri's GitHub issues and Discord (link on the docs site). Check the issue's date and Tauri version; v1-era answers are often wrong for v2.
9. **Still stuck? Make a minimal repro** and gather the pack in 14.5.

14.3 Which layer is it?

Symptom	Likely layer	First thing to try
<code>cargo</code> / <code>rustc</code> / <code>link.exe</code> not found, or setup commands fail	Environment (prereqs, PATH)	Section 1, reopen the terminal, <code>npm run tauri info</code>
Red Rust compile error (<code>error[E...]</code>)	Rust code or crate versions	Read the first error; <code>RUST_BACKTRACE=1</code> ; <code>cargo check</code>
Frontend build fails (<code>npm run build</code> error)	Frontend/bundler	Run <code>npm run build</code> alone and fix it there
Window opens blank in dev	Dev URL / dev server	<code>devUrl</code> vs actual port (<code>strictPort</code>), is the dev server running?
Window blank in built app, fine in dev	<code>frontendDist</code> path or static-export config	Check the output folder exists and has <code>index.html</code> ; section 13
Works in browser, fails in app (e.g. <code>window</code> /API errors)	Tauri bridge	WebView console; is the call client-side only? Section 13.9
<code>invoke</code> fails: "command not found"	Rust command not registered	Add it to <code>generate_handler![...]</code> , rebuild
Plugin call: "not allowed" / permission error	Capabilities	Section 6; <code>npm run tauri permission ls</code>
Build works, installer step fails	Bundling	Section 11 (Windows: C++ tools, VBSCRIPT for MSI)
Android-only failure	Android env/Gradle/signing	Section 9.7 table, <code>adb logcat</code>
iOS-only failure	Xcode / Apple signing / CocoaPods	Section 10.10 table, Xcode's console
Fails only in GitHub Actions	CI environment	Read the failing step; compare with 12.5 table; same command locally

14.4 Known problems, quick fixes

Problem	Fix
<code>cargo</code> / <code>rustc</code> not found	Reopen the terminal; confirm rustup finished
<code>link.exe</code> not found (Windows)	Install C++ Build Tools with "Desktop development with C++"
Port already in use	Stop the other dev server, or change the port in both your bundler config and <code>tauri.conf.json</code>
Plugin "not allowed"	<code>npm run tauri add <plugin></code> normally adds its default permission; anything beyond default goes in <code>src-tauri/capabilities/default.json</code>
Very slow first build	Normal; Rust compiles dependencies once
Strange errors after updating anything	Version mismatch check (14.2 step 4), then <code>cargo clean</code>
Rust update	<code>rustup update</code>
Tauri packages update	<code>npm update @tauri-apps/cli @tauri-apps/api</code>

14.5 When you ask for help (people or AI), send this pack

Good help depends on good input, and it saves back-and-forth (and AI credits). Paste:

1. The **exact command** you ran
2. The **first error** and ~20 lines around it (text, not a screenshot)
3. The output of `npm run tauri info`
4. **What changed** since it last worked (`git diff` summary)
5. Relevant config: `src-tauri/tauri.conf.json` (`build` section), and the framework config if it's a frontend issue
6. What you already tried

14.6 Bugs this guide doesn't cover (unknown unknowns)

Run 14.2 steps 1 to 5 anyway: they work for any error. If a fresh `create-tauri-app` project builds but yours doesn't, copy that project's config files (`tauri.conf.json` , `vite.config.*` , `src-tauri/Cargo.toml`) into yours one at a time until it breaks. That isolates the cause even when you don't understand it yet.

15. What this guide is sure about, and what to double-check

Everything here was compared with official Tauri docs and example workflows on 3 Oct 2026 unless listed below. These are the places where I was less sure, or where things move quickly.

Area	Confidence	What to double-check, and how
GitHub runner label (ubuntu-22.04)	Changed	GitHub began deprecating Ubuntu 22.04 runners on 17 Sep 2026 and retires them on 17 Apr 2027. Tauri's own example still uses 22.04, so this guide uses ubuntu-24.04 . ubuntu-latest is announced to move to Ubuntu 26.04 in Nov 2026, so pin the version if you want predictable builds. Check: https://github.com/actions/runner-images
Linux packages on Ubuntu 24.04	Medium	The guide uses the package names from Tauri's Debian prerequisites list (including libayatana-appindicator3-dev). I didn't run it on a 24.04 runner. If apt can't find a package, compare with the prerequisites page. Linux binaries built on a newer Ubuntu may not run on older distros
Android CI workflow	Medium	It's my own composition, not an official Tauri example. ANDROID_NDK_LATEST_HOME is confirmed in GitHub's runner image docs. Action major versions (setup-java@v5 , upload-artifact@v4) were not verified: if GitHub warns, bump to that action's latest major
APK output file name/path	Medium	Not stated in the official docs. The CLI prints the path when it finishes. To find it: find src-tauri/gen/android -name "*.apk" (Windows: Get-ChildItem -Recurse -Filter *.apk src-tauri\gen\android)
Android Gradle signing edit	Medium	Matches the official guide, last updated Aug 2025. If your generated build.gradle.kts looks different, find the buildTypes block in src-tauri/gen/android/app/build.gradle.kts and add the signing config there
Next.js setup	Medium	The official page says it's accurate as of Next.js 14.2.3. Newer Next.js versions may differ: check Next's own "Static Exports" docs. After npm run build , confirm an out/ folder with index.html exists
SvelteKit ssr = false tip	Medium	Community-known fix, not on the official page. The official recommendation is SPA mode via adapter-static with fallback: 'index.html'
Nuxt build command	Medium	The official checklist mentions nuxi build , but its example config uses npm run generate . I followed the config. After building, confirm dist/index.html exists
Qwik, Leptos, Trunk	Not checked	Follow their official setup pages (linked in 13.8)
Sections 4 and 5 (project structure, Rust command)	Medium	Standard v2 layout, not compared line by line. The create-tauri-app template is the reference: compare against a freshly generated project
--bundles nsis / msi	High	The CLI docs list bundle names for the host OS they were built on (Linux). On Windows run npm run tauri build -- --help to see them
MSI version-string rules	Removed	I couldn't confirm the old rule, so I left it out. If an MSI build rejects your version, use plain x.y.z
Windows code signing, macOS signing	Linked only	Follow the official signing pages
iOS section overall	Medium	Prerequisites, ios init / dev / build , --export-method app-store-connect , the .ipa path and the altool upload command match the official pages. I couldn't run any of it because iOS builds need a Mac
iOS --export-method names	Medium	Accepted values depend on your CLI and Xcode versions. Check npm run tauri ios build -- --help . The signing docs also mention an "ad-hoc" method
iOS Team ID setup (config key, env var, Xcode)	Medium	Comes from Tauri community discussions, and older posts conflict (one reported the config key not working while selecting the Team in Xcode did). Try the three options in 10.3 in order
iOS free-Apple-ID testing, Developer Mode, Safari inspector, xcode-select fix	General Apple knowledge	Not from the Tauri docs. Apple's menu names change between iOS/Xcode versions
iOS CI workflow	Low to	My own composition, untested. CI signing is the most failure-prone part: get one signed

(10.9)	medium	build working locally on a Mac first
iOS icon <code>--ios-color</code>	Medium	From the official App Store page; I added quotes around the <code>#</code> color so shells don't treat it as a comment
iOS encryption export compliance	Linked only	Follow the note on the official App Store page; requirements depend on your app
Rust, Node, Android Studio, Xcode versions	Moving	"Stable"/"LTS" change over time. <code>npm run tauri info</code> shows what you actually have

Rule of thumb: if this guide and the official Tauri docs disagree, the docs win. If a command fails in a way that isn't in section 14, use the process in 14.2 before changing anything.

16. Command cheat sheet

```

npm create tauri-app@latest      # new project
npm install                     # install JS deps
npm run tauri info               # environment health check (run this first when stuck)
RUST_BACKTRACE=1 npm run tauri dev # dev mode with Rust backtraces (PowerShell: $env:RUST_BACKTRACE=1 first)
npm run tauri build -- --debug   # packaged app with dev console enabled
npx tauri init                  # add Tauri to an existing frontend (section 13)
npm run tauri dev                # run in dev mode
npm run tauri build              # desktop installer
npm run tauri build -- --bundles nsis # Windows .exe installer only
npm run tauri add <plugin>       # add an official plugin
npm run tauri permission add <id> # allow a plugin permission
npm run tauri icon app-icon.png  # generate all icons
npm run tauri android init       # one-time Android setup
npm run tauri android dev        # run on phone/emulator
npm run tauri android build -- --debug --apk # quick installable APK
npm run tauri android build -- --apk        # release APK (needs signing)
npm run tauri android build -- --aab        # Play Store bundle
npm run tauri ios init           # one-time iOS setup (Mac only)
npm run tauri ios dev            # run on iOS Simulator / iPhone
npm run tauri ios build -- --export-method app-store-connect # .ipa for TestFlight / App Store
git tag app-v1.0.0 && git push origin main --tags # trigger GitHub release build
rustup update                    # update Rust

```

Useful links

- Docs: <https://v2.tauri.app>
- Prerequisites: <https://v2.tauri.app/start/prerequisites/>
- CLI reference: <https://v2.tauri.app/reference/cli/>
- Windows installer: <https://v2.tauri.app/distribute/windows-installer/>
- Android signing: <https://v2.tauri.app/distribute/sign/android/>
- iOS signing: <https://v2.tauri.app/distribute/sign/ios/>
- App Store distribution: <https://v2.tauri.app/distribute/app-store/>
- GitHub pipeline: <https://v2.tauri.app/distribute/pipelines/github/>
- Frontend frameworks: <https://v2.tauri.app/start/frontend/>
- Plugins: <https://v2.tauri.app/plugin/>
- Community help: Tauri Discord (linked from the docs)